

Documentation

Verifiable Credentials (VCs), FHIR and Data Exchange — Nuts-LSP

Author: Bernard Stibbe

Purpose: Overview of standards, Verifiable Credentials (VCs), FHIR usage, and trust chains within the Nuts-LSP Proof of Concept (PoC).

Version: 1.0.0

Date: 2025-11-28

0. Document Log

Version	Date	Author/ Editor	Description of Change	Remarks / Approved by
0.0.1	2025-11-12	B. Stibbe	Initial draft created.	—
0.0.2	2025-11-15	B. Stibbe	Updated context table.	Reviewed by Tom De Jong.
0.0.3	2025-11-18	B. Stibbe	Updated examples VC's and VP to W3C JWT format	
0.0.4	2025-11-18	B. Stibbe	Added OAuth 2.0 Token Exchange	
0.0.5	2025-11-19	B. Stibbe	Aligned with Nuts specs	
0.0.6	2025-11-24	B. Stibbe	Updated	This document was reviewed and commented by Ron van Holland and Marc van Gulik.
0.0.7	2025-11-27	B. Stibbe	Updated all VC's	Reviewed by PoC team (Itsos, AET, AoF)
0.0.8	2025-11-28	B. Stibbe	Updated all VC's and VP	Reviewed by Steven van der Vegt (Nuts)
1.0.0	2025-11-28	B. Stibbe	Final	Approved by the PoC Nuts-LSP team

Inhoud

0. Document Log	1
1. Introduction	3
2. Context: Twiin as Framework	5
3. Verifiable Credentials and Presentations	6
3.1 Standards and Protocols	6
3.2 JWT-Based Verifiable Credentials and Presentations – Fields, Meaning, and Example	7
3.2.1 JWT header (JWS)	7
3.2.2 JWT Payload Fields for VC's (W3C JWT-VC)	8
3.2.3 The VC Object Structure (W3C-Compliant)	9
3.2.4 Media type	9
3.2.5 Verification steps (high level)	10
3.3 JWT-VC Examples	11
3.3.1 HealthcareOrganizationVC (W3C JWT-VC)	12
3.3.2 PatientEnrollmentVC (W3C JWT-VC)	13
3.3.3 HealthcareProfessionalDelegationVC (W3C JWT-VC)	14
3.4 Standard VP Claims	15
3.4.1 VP-specific Claims	15
3.5 JWT-VP Example	15
3.6 Credential Issuance Flow (OIDC4VCI)	16
3.6.1 Issuer Discovery	16
3.6.2 Verifiable Credential Request	17
4. OAuth 2.0 Token Exchange	17
4.1 Requesting an Access Token using a VP	17
4.2 Parameter subject_token	17
4.3 Successful Response	18
5. FHIR Usage	18
6. Cryptography and Trust	18
7. Overview of VCs within the PoC	19
8. Credential Issuance and Trust Chain	19
9. Example Use Cases	19
10. Other Notes	19
11. Summary	20

1. Introduction

This document describes the technical standards and the use of Verifiable Credentials (VCs) and FHIR for exchanging patient and healthcare provider data between the Nuts network and the LSP (Landelijk Schakelpunt). To ensure secure, interoperable, and policy-compliant communication, all interaction between the two ecosystems is routed through the **GtK (Gevalideerd Twiin Knooppunt)**.

The GtK acts as a standardized access interface that separates the internal architecture of each domain (Nuts or LSP/AORTA) from the external party requesting or providing data.

There are **two distinct GtKs** involved:

1. GtK-Nuts

- Provides secure inbound and outbound communication for participants in the Nuts ecosystem.
- Enforces authorization rules based on the Verifiable Credentials and the underlying identifier/attribute models defined within Nuts.
- Translates external requests into the proper internal Nuts protocols and resource formats.

2. GtK-AORTA

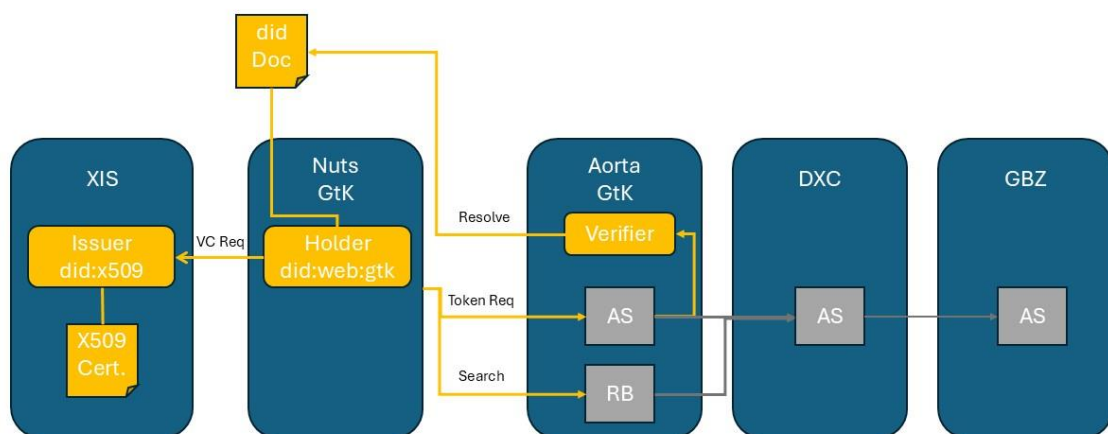
- Provides the standardized entry point into the LSP/AORTA infrastructure.
- Validates the authentication and authorization metadata (such as tokens, signatures, or VCs) according to LSP/AORTA trust frameworks.
- Ensures consistent routing to the appropriate LSP services, such as medication, professional/patient records, or consent services.

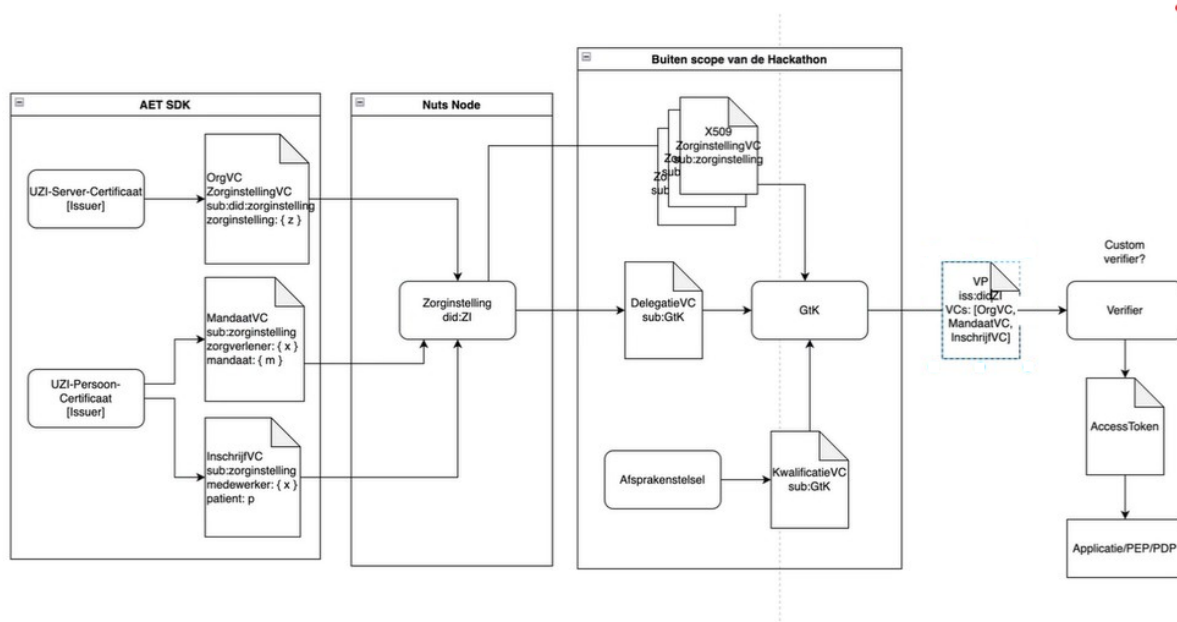
By placing a GtK on each side, both ecosystems can evolve independently while maintaining a stable, secure interface for cross-domain data exchange.

The Nuts–LSP bridge therefore consists of:

- **A Verifiable Credential–based authorization model,**
- **FHIR-based clinical data structures,** and
- **Two GtKs that mediate, validate, and convert the communication flows.**

This approach ensures that trust, authorization, and interoperability remain consistent across the entire end-to-end exchange path, regardless of which party initiates the communication.





The diagram shows how Verifiable Credentials flow from UZI certificates, through the AET SDK and the Nuts Node, into the GtK (Gevalideerd Twiin Knooppunt), where they are validated and used for authorization.

- The **AET SDK** issues three VCs based on UZI server and person certificates.
- The **Nuts Node** links these credentials to the organization's decentralized identifier (did:ZI) and makes them available for cross-domain use.
- The **GtK** receives these VCs, combines them with X.509-based organizational credentials, and applies the Twiin trust and authorization rules.
- A **Verifier** validates the final Verifiable Presentation and issues an Access Token that applications can use to access LSP/AORTA services.

In short, the diagram illustrates the trust chain from UZI certificates to VCs, through the Nuts and GtK infrastructure, ending in a verifiable authorization decision for healthcare applications.

Objectives:

- Ensure interoperability with existing holders, issuers, and verifiers.
- Avoid creating a proprietary "Nuts-LSP VC format"; align with recognized international standards.
- Prepare for future implementations such as DEZI.

2. Context: Twiin as Framework

- Twiin acts as the overarching framework for standards and architectural choices (network connections, FHIR R4, GTK definitions).
- GTKs (Validated Twiin Nodes): Concrete implementations under the Twiin framework.
- Twiin provides the framework; standards and protocols derive from it.

Used standards:

Domain	Standard/Technology	Description
Data Exchange	FHIR R4	HL7 FHIR Release 4, basis for healthcare data exchange.
Transport & Security	RESTful API, mTLS 1.3, OAuth2.0, NEN7510	Authentication and authorization for API traffic

Domain	Standard/ Technology	Description
Identification	URA, UZI-number, BSN	Provider and patient identity
VC Protocols	OIDC4VCI	Verifiable credential issuance
Retrieving Medication Data	MP 9.3 Nictiz: FHIR search for Administration Agreements	The integration in the PoC will initially focus on retrieving instances of medication agreements in accordance with the Nictiz MP 9.3 information standard. Format: GET [base]/MedicationDispense?category=http://snomed.info/sct 422037009

3. Verifiable Credentials and Presentations

3.1 Standards and Protocols

Domain	Standard	Description
VC Data Model	W3C Verifiable Credentials Data Model v2.0	Defines structure, semantics, and lifecycle of credentials. Link: https://www.w3.org/TR/vc-data-model/
JWT-VC Representation	W3C JWT-VC	We use the JWT-based Verifiable Credential (VC-JWT) representation defined by the W3C Verifiable Credentials Data Model v1.1 (03-03-2022). Link: https://www.w3.org/TR/vc-jwt/
Issuance Protocol	Out of scope for the hackathon: OpenID for Verifiable Credential Issuance (OID4VCI)	Credential request and issuance (issuer ↔ holder (wallet))
Presentation Protocol	Profile on RFC 7523	Credential presentation from holder to verifier Link: https://build.fhir.org/ig/nuts-foundation/nl-generic-functions-ig/branches/authentication/GFI-004.html
Decentralized Identifiers	W3C DID v1.0	Decentralized Identifiers (DIDs) are a type of identifier for verifiable, decentralized digital identity. Link: https://www.w3.org/TR/2022/REC-did-core-20220719/
did:x509	A working draft for the specification for the did:x509 DID method.	Link: https://trustoverip.github.io/tswg-did-x509-method-specification
did:web	DID method to resolve public keys using the DNS system secured by TLS	Link: https://w3c-ccg.github.io/did-method-web/

Domain	Standard	Description
Digital Signature	RFC 7515 (JWS) / RFC 7518 (algorithms)	Cryptographic signing of the JWT payload (RS256/ES256)

3.2 JWT-Based Verifiable Credentials and Presentations — Fields, Meaning, and Example

A W3C **JWT-based Verifiable Credential (JWT-VC)** is a Verifiable Credential represented as a signed JSON Web Token (JWT) using the W3C Verifiable Credentials Data Model v2.0 and the JWS-based Data Integrity proof method.

Unlike SD-JWT VC, the W3C JWT-VC format does **not** support selective disclosure and does **not** use hashed claim placeholders or disclosure bundles. All credential attributes are included directly and clearly within the JWT.

The W3C specification defines:

- Which fields must appear in the JWT header and payload,
- How the vc object is structured (including @context, type, and credentialSubject),
- How verifiers should validate signatures, timestamps, issuer identifiers, and credential content.

A JWT-VC consists of:

1. **A standard JWS header** (e.g., alg, typ, kid);
2. **A standard JWT payload** containing the issuer, subject, timestamps, and a vc object;
3. **A JWS signature** generated using the issuer's key referenced from its DID document.

This structure ensures interoperability, clarity of semantics, and compatibility with existing W3C VC holders, issuers, and verifiers without relying on IETF selective-disclosure mechanisms.

3.2.1 JWT header (JWS)

The header contains standard JOSE fields plus optional X.509 certificate references.

Field	Meaning	Example / Comment
alg	Cryptographic algorithm used to sign (or encrypt) the JWT	"RS256"
typ	Type of token	"JWT"
kid	Key identifier for the signing key. Required if multiple keys exist; recommended to	"did:x509:0:sha256:WE4P5dd8DnLHskyHaljhp4udlkF9LqoKwCvu9gl38jk::subject:O:OLVG%20Oost::san:otherName:2.16.528.1.1003.1.3.5.5.5:1-987654321-S-12345678-00.000-00000000"

File Id	Meaning	Example / Comment
	bind the VC to a specific verification key.	
x5c	X.509 certificate chain (base64 DER). Used for signature verification in X.509-based VCs	Inline certificate chain for key validation. Used in X.509-based Verifiable Credentials to include the issuer's certificate chain for signature verification.

Below is an example of an X509Credential issued by a did:x509. The code is a JWT header.

```
{
  "alg": "RS256",
  "typ": "JWT",
  "x5c": [
    "MIID...<base64 DER of leaf-certificate >...",
    "MIID...<base64 DER of intermediate CA-certificate>...",
    "MIID...<base64 DER of root CA-certificate>..."
  ],
  "kid":
  "did:x509:0:sha256:WE4P5dd8DnLHskyHaIjhp4udlkF9LqoKwCvu9gl38jk::subject:O:OLVG%20Oost::san:otherName:2.16.528.1.1003.1.3.5.5.5:1-987654321-S-12345678-00.000-00000000"
}
```

The JWT Header contains the X.509 certificate chain and uses a SHA-256 thumbprint as the Key ID of the intermediate CA. Pinning to an intermediate CA means you explicitly bind the did:x509 to that intermediate certificate's fingerprint and you must update the DID whenever the intermediate CA changes or expires. See also the specification in <https://trustoverip.github.io/tswg-did-x509-method-specification/#operations>.

Note: The did:x509 method allows you to pin either to an intermediate CA or to the root CA. Pinning to the intermediate CA provides a tighter binding but results in shorter DID lifetimes, because the DID must be updated whenever the intermediate certificate rolls over or expires. Pinning to the root CA creates a more stable DID, as root certificates have long validity periods and intermediate certificates can change over time without requiring a DID update. In all cases, standard X.509 path validation still applies: the leaf certificate, all intermediates, and the root must form a valid chain at the time of verification.

3.2.2 JWT Payload Fields for VC's (W3C JWT-VC)

Below is a complete table of the **minimal used payload fields** in a W3C JWT-VC, including required and optional fields.

JWT Payload Field Table

Field	Meaning	Example / Comment
iss	Identifier of the issuer (typically a DID).	"iss": "did:<issuer>:xxxxx"
sub	Identifier of the subject the credential refers to.	"sub": "did:web:zorgaanbieder.example"
jti	Unique identifier for the credential (JWT ID).	"jti": "urn:uuid:123e4567-e89b-12d3-a456-426614174000"
iat	Issued-at timestamp (seconds since epoch).	"iat": 1730876100
nbf	Not-before timestamp; credential becomes valid at this time.	"nbf": 1730876100
exp	Expiration timestamp; credential must not be accepted after this time.	"exp": 1762412100
vc	The Verifiable Credential object containing the W3C fields (@context, type, credentialSubject).	Full structure shown below

3.2.3 The VC Object Structure (W3C-Compliant)

The vc object always contains:

Field	Meaning	Example / Comment
@context	Defines semantics and vocabulary for interpreting the credential.	"@context": ["https://www.w3.org/ns/credentials/v2"]
type	An array defining the credential type(s). Must include "VerifiableCredential".	"type": ["VerifiableCredential", "HealthcareOrganizationVC"]
credentialSubject	The domain-specific attributes about the subject of the credential.	The credentialSubject.id is optional according to W3C. It MUST NOT contain sensitive identifiers such as a BSN. Other attributes (e.g., organizationName, uziNumber, authorizationRole) may be included depending on the credential type.

3.2.4 Media type

JWT-based Verifiable Credentials use the media type application/jwt.

Implementations and transport layers SHOULD use this content type when transmitting a JWT VC directly.

When a JWT VC is transported using **OAuth 2.0 mechanisms** (e.g., Token Exchange or POST requests to a token endpoint), the transport protocol **requires** the use of:

Content-Type: application/x-www-form-urlencoded

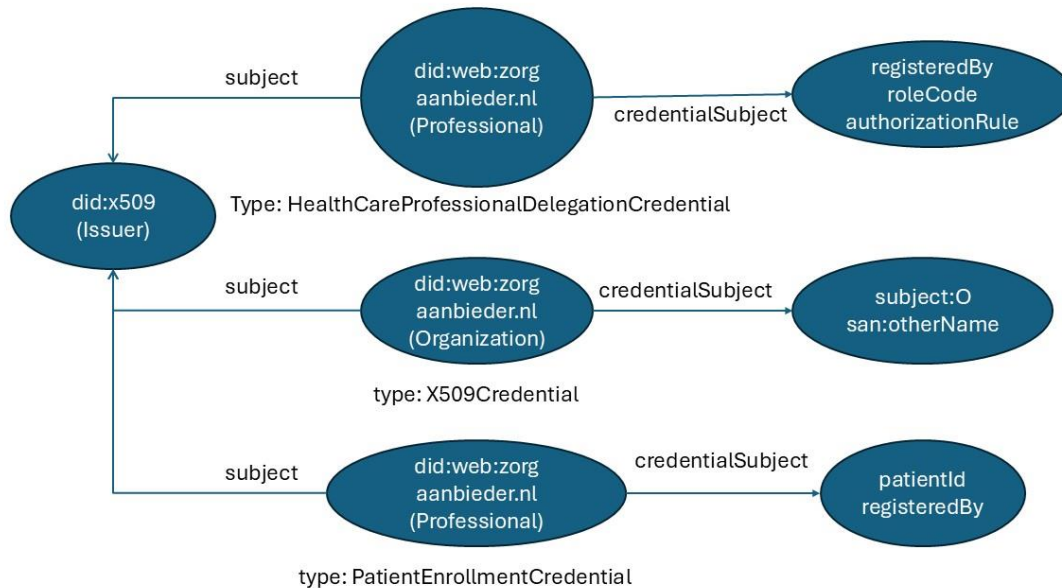
This requirement originates from OAuth 2.0, not from the VC-JWT specification itself.

3.2.5 Verification steps (high level)

When a verifier receives a JWT-based Verifiable Credential or Verifiable Presentation, it typically **SHOULD**:

1. **Verify the JWS signature**
Check the alg header and validate the signature using the issuer's public key and/or certificate chain.
2. **Parse the JWT payload**
Extract claims, including the embedded vc or vp object.
3. **Validate the credential type and metadata**
Using the type field and the contents of the vc object, verify the credential type, mandatory domain-specific claims, claim semantics, and any additional validation rules defined by the credential's schema or trust framework.

3.3 JWT-VC Examples



For this Proof of Concept (PoC), we use two DID methods (**did:x509** and **did:web**), one issuer, one holder, and three Verifiable Credentials (VCs):

- **HealthcareOrganizationVC (X509 Credential)**
- **PatientEnrollmentVC ((PatientEnrollment Credential)**
- **HealthcareProfessionalDelegationVC (HealthcareProfessionalDelegation Credential)**

The diagram illustrates the relationships between these three VCs, showing how their *issuer*, *subject*, and *credentialSubject* fields are connected through the **did:x509** method.

- **did:x509** — used to express identities derived from X.509 certificates
- **did:web** — used to express identities controlled by the healthcare provider's domain (e.g., *zorgaanbieder.nl*). See also the DID Web specification: <https://w3c-ccg.github.io/did-method-web/>.

In the context of Verifiable Credentials:

- The **issuer** is the entity that creates and cryptographically signs the credential, asserting that the claims it contains are valid.
- The **holder** is the entity that receives the credential and can later present it as proof of those claims.

A **did:x509** can represent either an individual or an organization, because X.509 certificates can be issued to both types of entities.

Caching of DID documents

When resolving a DID—especially **did:web**—the DID document is retrieved via a standard HTTPS GET request. To ensure efficient resolution and to avoid unnecessary network traffic, the DID document response may include HTTP caching directives. For example:

```
Cache-Control: public, max-age=3600
```

This allows verifiers and intermediaries to cache the DID document for up to one hour, reducing load on the hosting server and improving performance, while still ensuring timely updates when the DID document changes.

3.3.1 HealthcareOrganizationVC (W3C JWT-VC)

Issuer

- The VC is issued by an entity represented as **did:x509:(Issuer)**. This DID correspond to the *leaf certificate* of a healthcare provider issued under the **UZI-register/PKloverheid** hierarchy.

Subject

- The subject is **did:web:zorgaanbieder.nl** which identifies the healthcare provider.

Credential Subject

- id — DID of the healthcare organization
- subject:O — organization name
- sanOtherName — derived from UZI Private Server Certificate

```
{
  "iss": "did:x509:0:sha256:WE4P5dd8DnLHskyHaIjhp4udlkF9LqoKwCvu9gl38jk::subject:O:
Zorginstelling%20Example::san:otherName:2.16.528.1.1003.1.3.5.5.5:1-987654321-S-12345678-
00.000-000000000",
  "sub": "did:web:zorgaanbieder.nl",
  "iat": 1731379200,
  "nbf": 1731379200,
  "exp": 1762915200,
  "jti": "urn:uuid:0d1600bb-4e9b-4d89-b77f-002ff1c1a9ad",
  "vc": {
    "@context": [
      "https://www.w3.org/2018/credentials/v1"
    ],
    "type": ["VerifiableCredential", "X509Credential"],
    "credentialSubject": {
      "id": "did:web:zorgaanbieder.nl",
      "subject:O": "Zorgaanbieder Example"
      "san:otherName": "2.16.528.1.1003.1.3.5.5.5:1-987654321-S-12345678-00.000-000000000"
    }
  }
}
```

Note: "san:otherName"

This refers to the SubjectAltName.OtherName attribute, which carries specific meaning in the healthcare sector. Below is how the subjectAltName.otherName is structured in certificates issued by the UZI register.

PKloverheid specifies a subjectAltName.otherName using an OID-like structure: <OID_CA>:<Subject_ID>

- <OID_CA> and <Subject_ID> are separated by a colon (:).
- <OID_CA> represents the OID of the issuing CA, following the structure <PKloverheid>.<Domain>.<TSP>.<CA>.
 - For the UZI Register 'Zorgverlener' CA (the mandator), this is: **2.16.528.1.1003.1.3.5.5.2**
 - For the UZI Register 'Medewerker op Naam' CA, this is: **2.16.528.1.1003.1.3.5.5.3**
 - For the UZI Register Server CA, this is: **2.16.528.1.1003.1.3.5.5.5**
- <Subject_ID> is a unique identifier within the TSP domain. The UZI register defines it to include multiple numbers that have meaning within the healthcare sector and uniquely identify the subject as a healthcare professional within a given subscriber.

SubjectAltName.otherName value (<Subject_ID>):

The <Subject_ID> in the UZI register is a composite field, with components separated by hyphens:

<Subject_ID> = <version>-<UZI_number>-<pas_type>-<subscriber_number>-<role>-<AGB_code>

Example: 1-987654321-S-12345678-00.000-00000000

The HealthcareOrganizationVC is signed with the UZI-register Private Server certificate of the Healthcare Provider (UZI-register Private Server CA G1) with algorithm: RSA-256. The issuer (a DID based on the CA) and the signer are linked through the leaf certificate.

3.3.2 PatientEnrollmentVC (W3C JWT-VC)

Issuer

- This VC is issued by **did:web:zorgaanbieder.nl**. The healthcare provider signs this credential using its **did:web** DID and corresponding domain-controlled private key.

Subject

- The subject is **did:web:zorgaanbieder.nl** which identifies the healthcare provider.

Credential Subject

- id – DID of the healthcare organization.
- patientId - The Dutch Citizen Service Number (BSN) of the patient. Prefixed by the fhir system. This is sensitive personal data and should only be included in the VC payload for point-to-point verification, never as a publicly resolvable identifier.
- registeredBy - Identifier of the staff member who performed the registration. Prefixed by the fhir system. Usually the UZI identifier, should match with the UZI-number field of the san:otherName field from the issuer string. (the issuer string should be verified with the certificate fields).

The fields **registeredAt** and **registeredBy** point back to entities described in the two x.509 VCs (professional and organization).

```
{
  "iss":
  "did:x509:0:sha256:WE4P5dd8DnLHskyHaIjhp4udlkF9LqoKwCvu9gl38jk::san:otherName:2.16.528.1.1003.1.3.5.5.3:1-882334175-M-12345678-00.000-00000000",
  "sub": "did:web:zorgaanbieder.nl",
  "iat": 1730876000,
  "nbf": 1730876100,
  "exp": 1762412100,
  "jti": "urn:uuid:32fb679f-1875-4a78-b8be-27e13f40ef49",
  "vc": {
    "@context": [
      "https://www.w3.org/ns/credentials/v1"
    ],
    "type": [
      "VerifiableCredential",
      "PatientEnrollmentCredential"
    ],
    "credentialSubject": {
      "id": "did:web:zorgaanbieder.nl",
      "patientId": "http://fhir.nl/fhir/NamingSystem/bsn|87654321",
      "registeredBy": "http://fhir.nl/fhir/NamingSystem/uzi-nr-pers|882334175",
    }
  }
}
```

The PatientEnrollmentVC is signed with the UZI-Pas certificate of the Healthcare Professional (UZI-Pas Zorgverlener of Medewerker op Naam CA G3) with algorithm: RSA-256.

A did:web identifier is always resolved by retrieving the DID Document via HTTPS from the domain itself. For example, the DID **did:web:zorgaanbieder.nl** resolves to: **https://zorgaanbieder.nl/.well-known/did.json**

(*) Note:

The resolver fetches the document with a request such as:

```
GET /.well-known/did.json HTTP/1.1
Host: zorgaanbieder.nl
Accept: application/json
```

Once retrieved, the DID Document should be checked to ensure:

- The JSON is valid.

- It contains "id": "did:web:zorgaanbieder.nl".
- It includes at least one verificationMethod with a public key.

These checks guarantee that the DID Document is authentic and that the public key can be used to verify the PatientEnrollmentVC signature.

3.3.3 HealthcareProfessionalDelegationVC (W3C JWT-VC)

A credential in which describes a delegation given from a healthcare professional to other personal of a care provider.

Specification of the original SAML Mandaattoken AORTA v.8.2.0.0: <https://www.aorta-lsp.nl/media/downloads/implementatiehandleiding-mandaattoken-aorta-v8200pdf/download>

Issuer

- The VC is issued by an entity represented as **did:x509:(Issuer)**. This DID corresponds to the *leaf certificate* of a healthcare professional issued under the **UZI Register / PKIoverheid** hierarchy by selecting the leaf from a chain by a query that should match the given fields.

Subject

- The sub value is **did:web:zorgaanbieder.nl**. This DID identifies the professional in the healthcare provider's own namespace.

Credential Subject

- **id** — The unique identifier of the subject for this credential. In this case, it represents the professional in the healthcare provider's own namespace.
- **RegisteredBy**: the identification of the healthcare professional, prefixed by the system (UZI). Must match the corresponding position in the san:othername of the the issuer.
- **RoleCode**: the role code of the healthcare professional. Must match the corresponding position in the san:othername of the the issuer.
- **AuthorizationRule**: internal rule of the care provider.

```
{
  "iss":
    "did:x509:0:sha256:WE4P5dd8DnLHskyHaIjhp4udlkF9LqoKwCvu9gl38jk::san:otherName:2.16.528.1.1003.1.3.5.5.2:1-882354372-Z-12345678-01.015-00000000",
  "sub": "did:web:zorgaanbieder.nl",
  "iat": 1541493724,
  "nbf": 1541493724,
  "exp": 1573029723,
  "jti": "urn:uuid:9877e792-d51b-46a1-9d37-c748e531c57a",
  "vc": {
    "@context": [
      "https://www.w3.org/ns/credentials/v1"
    ],
    "type": [
      "VerifiableCredential",
      "HealthCareProfessionalDelegationCredential"
    ],
    "credentialSubject": {
      "id": "did:web:zorgaanbieder.nl",
      "registeredBy": "http://fhir.nl/fhir/NamingSystem/uzi-nr-pers|882354372",
      "roleCode": "01.015",
      "authorizationRule":
        "https://goedbeheerdziekenhuis/autorisatieregels/medicatiecontext/v2"
    }
  }
}
```

The HealthcareProfessionalDelegationVC is signed with the UZI-Pas certificate of the Healthcare Professional (UZI-Pas Zorgverlener CA G3) with algorithm: RSA-256.

Notes

- All timestamps (iat, nbf, exp) and jti should always be included in production credentials for validity and uniqueness.

3.4 Standard VP Claims

A W3C JWT Verifiable Presentation (JWT-VP) is a cryptographically signed object that presents one or more Verifiable Credentials (VCs) to a verifier. Using JWT allows standard cryptographic validation.

Field	Meaning	Example / Comment
iss	Issuer of the VP; usually the holder presenting the credentials.	"did:web:gtk:nuts"
aud	Audience; the verifier intended to receive the VP.	"did:web:gtk:aorta."
iat	Issued At; timestamp when VP was created.	1700300000
nbf	Not Before; VP should not be considered valid before this time.	1700300000
exp	Expiration; optional but recommended to limit validity.	1700386400
jti	JWT ID; unique identifier for the VP.	"vp-12345"

3.4.1 VP-specific Claims

Field	Meaning	Example / Comment
vp	Main object containing the presentation data.	{ "type": ["VerifiablePresentation"], "verifiableCredential": [...] }
vp.type	Type of the presentation (always includes "VerifiablePresentation").	["VerifiablePresentation"]
vp.verifiableCredential	Array of embedded VCs (JWT-VCs)	["eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."]
nonce	Used to prevent replay attacks in some protocols.	"1234567890"

3.5 JWT-VP Example

In the Proof of Concept, we decided that a GtK is functionally identical to a healthcare provider. This also means that the holder of the Verifiable Presentation (VP) is the healthcare provider itself. Since a VP must always be signed by the holder, the `iss` (issuer) claim in the JWT-VP must represent the healthcare provider's identity.

Because the healthcare provider controls its own domain, the correct DID method to express this identity is **did:web**, as defined in the DID Web specification (<https://w3c-ccg.github.io/did-method-web/>). Therefore, the `iss` value should not be a generic GtK identifier, but the DID of the healthcare provider's domain. This allows the verifier (e.g., the LSP) to resolve the DID directly at <https://<domain>/.well-known/did.json> and obtain the public keys needed to verify the VP signature.

```
{
```

```

"iss": "did:web:zorgaanbieder.nl",
"jti": "urn:uuid:3978344f-8596-4c3a-a978-8fcaba3903c5",
"aud": "did:web:lsp.nl"
"nbf": 1541493724,
"iat": 1541493724,
"exp": 1541493754,
"vp": {
  "type": ["VerifiablePresentation"],
  "verifiableCredential": [
    "base64(HealthcareOrganizationVC)",
    "base64(PatientEnrollmentVC)",
    "base64(HealthcareProfessionalDelegationVC)"
  ]
}
}

```

This aligns the VP's issuer with the actual holder (the healthcare provider) and ensures a consistent and verifiable trust model within the PoC.

To prevent replay attacks, the time between iat and exp should be very short, like 5 to 30 seconds.

3.6 Credential Issuance Flow (OIDC4VCI)

Not in scope, for the PoC, we will be using copy paste between the AET issuer and the wallet.

The issuance of **Verifiable Credentials (VCs)** follows the **OpenID for Verifiable Credential Issuance (OIDC4VCI, draft v1.0)** standard, which is an extension of the **OpenID Connect** and **OAuth 2.0** protocols.

This standard defines how an **issuer** can issue a credential to a **holder** or **wallet**, using verifiable identities (such as **DIDs** or certificates) and a secure token exchange.

The **OIDC4VCI flow** consists of three main phases:

1. **Issuer Discovery**
2. **Token Request** (OAuth 2.0 Client Credentials Flow)
 - a. **Not required for machine-to-machine flows**, as the client is already authenticated.
3. **Verifiable Credential Request**

3.6.1 Issuer Discovery

The issuer publishes a `/.well-known/openid-credential-issuer` document. This document specifies:

- Which types of credentials are supported
- Which cryptographic algorithms are used
- How a client (GTK or healthcare provider) can request a credential

Example:

```

{
  "credential_issuer": "https://issuer.zorgid.example",
  "credentials_supported": [
    {
      "id": "HealthcareOrganizationVC",
      "format": "jwt_vc_json",
      "cryptographic_binding_methods_supported": ["did:x509"],
      "cryptographic_suites_supported": ["RS256"]
    },
    {
      "id": "PatientEnrollmentVC",
      "format": "jwt_vc_json",
      "cryptographic_binding_methods_supported": ["did:x509"],
      "cryptographic_suites_supported": ["RS256"]
    }
  ]
}

```



```
{
  "id": "HealthcareProfessionalDelegationVC",
  "format": "jwt_vc_json",
  "cryptographic_binding_methods_supported": ["did:x509"],
  "cryptographic_suites_supported": ["RS256"]
}
```

Key points:

- **VC issuer** → X.509 DID (did:x509)
- **VC subject** → Healthcare provider (did:web)
- **VP issuer / holder** → Healthcare provider (did:web)
- **Verifier** → Resolves VP via did:web, then validates VCs using did:x509 signature

This aligns with **OIDC4VCI discovery rules**, where an issuer can support multiple credential types under the same issuer URL.

3.6.2 Verifiable Credential Request

```
POST /credential/request HTTP/1.1
Host: issuer.zorgid.example
Content-Type: application/json
Authorization: Bearer <access_token>
```

```
{
  "type": "urn:nl:vc:<XYZ>VC",
  "format": "jwt_vc_json",
  "credentialSubject": {
    "id": "did:<xyz>:<234567821>",
    ...
  }
}
```

Notes

- Authorization header contains an OAuth 2.0 access token obtained via the **Client Credentials Flow** or wallet authentication.
- *Optional* a cnf.jwk_thumbprint field binds the VC to the holder's public key, ensuring **only the intended holder can present it**.
- credentialSubject.id is always the DID of the entity that the VC represents (organization, professional, patient, or GTK).
- The type field corresponds exactly to the VC type in the issuer's discovery document.

4. OAuth 2.0 Token Exchange

This is the **IETF / OAuth standard** that allows a client to submit an external token (like a JWT Verifiable Presentation) to the token endpoint and receive an **Access Token** in return.

4.1 Requesting an Access Token using a VP

- Any "security token" can be submitted.
- A **JWT-VP** is technically a JWT, so it can be used here.

Important: Both examples of RFC 7523 and RFC 8693 require that the JWT (whether it is an assertion, VC, or VP) is always included in the **HTTP POST body**, encoded as application/x-www-form-urlencoded. It is **not** placed in an HTTP header.

4.2 Parameter subject_token

- The VP (in JWT format) can be placed here as an assertion

- To stay aligned with the TA NP, we choose RFC 7523, where a single assertion (a statement expressed as a JWT) is included as a parameter when requesting an access token.

Example request from RFC 7523 – JWT Bearer Token Request:

```
POST /token
Content-Type: application/x-www-form-urlencoded
grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer
&assertion=<JWT>
&client_id=<GtK-id>
```

*Formally, RFC 7523 does **not** explicitly support exchanging arbitrary token types (such as VCs and VPs). RFC 8693 does allow this.*

Example request from RFC 8693:

```
POST /token
Content-Type: application/x-www-form-urlencoded

grant_type=urn:ietf:params:oauth:grant-type:token-exchange
&subject_token=<JWT>
&subject_token_type=urn:ietf:params:oauth:token-type:jwt
```

Replace <JWT> with a JWT-encoded Verifiable Presentation.

Note: For the Nuts-LSP PoC, we will initially use RFC 7523, where the VP is sent via an assertion to obtain an access token.

4.3 Successful Response

- If the submitted token (VP) is accepted, the server returns an **Access Token**.

5. FHIR Usage

- **Model:** HL7 FHIR R4.
- **API:** REST API with authentication via Verifiable Credentials (VCs) or tokens.
- **Profiling:** Nictiz FHIR profiles for compatibility
- **Example:** PatientEnrollmentVC linked to a BSN

6. Cryptography and Trust

- **Signature:** JWS using RS256 according to RFC 7515 & RFC 7518.
- **DID Methods:** W3C DID Core 1.0 (did:web, did:x509).
- **Trust Chain:** PKloverheid certificates (UZI/Dezi) or Nuts root CAs.

7. Overview of VCs within the PoC

VC-type	Issuer	Subject	Key Attributes	VP Holder
PatientEnrollmentVC	did:x509:UZl-register-Zorgverlener of Medewerker op Naam	did:web:zorgaanbieder.nl	id, patientId, registeredBy	did:web:zorgaanbieder.nl
HealthcareProfessionalDelegationVC	did:x509:UZl-register-Zorgverlener	did:web:zorgaanbieder.nl	id, registeredBy, roleCode, authorizationRule	did:web:zorgaanbieder.nl
HealthcareOrganizationVC	did:x509:UZl-register-Private-Server	did:web:zorgaanbieder.nl	id, subjectO, healthcareOrganizationId, sanOtherName	did:web:zorgaanbieder.nl

Notes:

- Holder for Verifiable Presentations = Healthcare Provider GTK (Nuts).
- TransactionVC is not used as a VC; it is only a label for messages.
- Subject and holder do not need to be the same.

8. Credential Issuance and Trust Chain

1. Source Issuer: CIBG (UZI card / X.509 PKIO certificate).
2. Operational Issuer: Healthcare provider, based on an authenticated UZI identity.
3. Chain: CIBG → UZI certificate → Healthcare Professional → Healthcare Provider → VC.

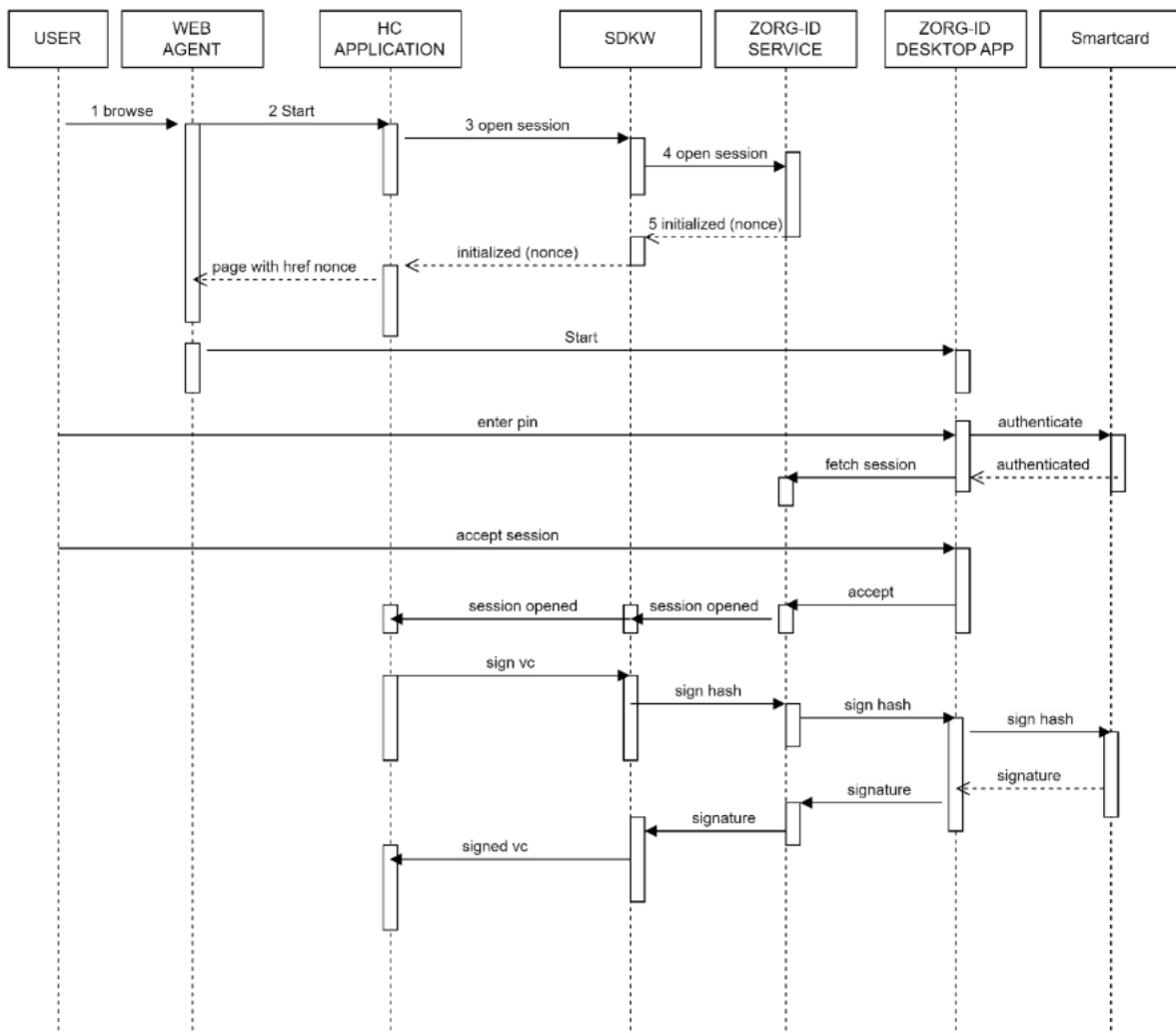
9. Example Use Cases

- **PatientEnrollmentVC**: Confirms the enrollment of a patient by a healthcare provider. See 3.3.2.
- **HealthcareProfessionalDelegationVC**: Confirms delegation of roles within a HealthcareProvider. See 3.3.3.
- **HealthcareOrganizationVC**: Verifies attributes of a healthcare organization. See 3.3.1.

10. Other Notes

- Documentation will be maintained on Confluence.
- Code, examples and templates are maintained on GitHub
 - <https://github.com/nuts-foundation/poc-lsp-nuts>
- Comparison of Nuts RFC0023 with DID-X509 specification is required.

- VCs must remain compatible with future implementations such as DEZI.



11. Summary

- Goal: A standardized, verifiable trust layer between Nuts and the LSP.
- Standards: W3C VC v2.0, OID4VCI, JWT-VC, JWT-VP, OAuth 2.0, FHIR R4, PKIoverheid/Nuts certificates.
- Key Concepts: Separation of identity (authentication), attributes (VC), subject/holder, and interoperability.